One rotation

How SHA-0 and SHA-1 were broken, and what the difference between them was worth

An appendix to the `shavar` repository

Abstract

SHA-0 and SHA-1 are the same function. They differ in one place: SHA-1 rotates the message-expansion feedback left by a single bit, and SHA-0 does not. That rotation is the whole of the change NIST made in 1995, without explaining why, and it is worth about 2^{24} in attack cost.

This document explains how both functions were broken, and the accompanying code in `broken/` runs the argument rather than describing it: the structural weakness is *proved* in Lean, the local-collision probabilities are *measured* rather than quoted, the space of usable difference patterns is *enumerated* exhaustively, and real collisions in reduced-round SHA-0 are *found* in milliseconds. Full SHA-0 and SHA-1 collisions are not reproduced here, and §7 says so plainly.

Contents

1 Two functions, one rotation

SHA-0 is the hash function published by NIST in 1993 as FIPS 180 [1]. In 1995 NIST withdrew it and published FIPS 180-1 [2], whose function differs in exactly one expression. The new function was called SHA-1; the old one acquired the retrospective name SHA-0. NIST gave no public reason. The name *SHA-0* appears nowhere in either standard.

Both functions hash a message in 512-bit blocks to a 160-bit digest. Both pad identically, use the same five-word initial value, the same four round constants, the same three round functions and the same 80 rounds. Both begin by expanding the sixteen 32-bit words of a block into eighty

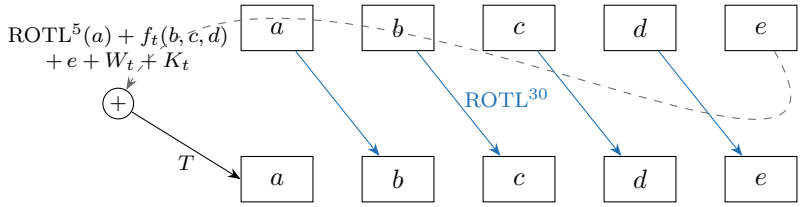

Figure 1: One round, identical in both functions. Four of the five register updates are copies; the only arithmetic is the new a, and the only displacement is the ROTL^{30} on the way into c. That rotation is why a difference introduced at bit i has to be cancelled at bit $i+30$ three rounds later, which is the shape of the local collision in Figure 2.

schedule words $W_0 \dots W_{79}$. The first sixteen are the message words themselves. The rest are where the two functions part:

$$\text{SHA-0:} \quad W_t = W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16} \tag{1}$$

$$\text{SHA-1:} \quad W_t = \mathrm{ROTL}^1(W_{t-3} \oplus W_{t-8} \oplus W_{t-14} \oplus W_{t-16}) \tag{2}$$

That is the entire difference. In `broken/c/sha01.c` it is one expression with the rotation amount as a parameter, so that SHA-0 is the rotation by zero:

```
uint32_t f = W[t-3] ^ W[t-8] ^ W[t-14] ^ W[t-16];
W[t] = sha01_rotl(f, (unsigned)v);   /* v = 0 -> SHA-0, v = 1 -> SHA-1 */
```

1.1 The round

Both functions keep five working registers a, b, c, d, e and run

$$T = \mathrm{ROTL}^5(a) + f_t(b,c,d) + e + W_t + K_t \pmod{2^{32}} \tag{3}$$
$$(a,b,c,d,e) \leftarrow (T,\ a,\ \mathrm{ROTL}^{30}(b),\ c,\ d)$$

with f_t chosen by round range:

rounds	$f_t(b,c,d)$	
0–19	$(b \wedge c) \oplus (\neg b \wedge d)$	Ch, nonlinear
20–39	$b \oplus c \oplus d$	Parity, **GF(2)-linear**
40–59	$(b \wedge c) \oplus (b \wedge d) \oplus (c \wedge d)$	Maj, nonlinear
60–79	$b \oplus c \oplus d$	Parity, **GF(2)-linear**

Half the rounds have a linear f. §3 shows what that costs.

2 Why the expansion is the weak part

Both expansions are linear over GF(2): the expansion of an XOR difference is the XOR difference of the expansions. That is what lets an attacker reason about differences at all, and it is true of both functions.

The difference is *what the linearity is over*. In (1) no bit ever changes position. Bit 7 of every W_t depends only on bit 7 of earlier words. SHA-0's expansion is therefore thirty-two independent, identical copies of one scalar recurrence

$$x_t = x_{t-3} \oplus x_{t-8} \oplus x_{t-14} \oplus x_{t-16} \tag{4}$$

over GF(2). In (2) the rotation carries bit i into position $i+1$ at every step, stitching the thirty-two copies into a single recurrence on 512 bits.

2.1 Measured

`broken/attack/expansion.c` feeds a *one-bit* difference through both expansions and counts where it goes. Because the expansion is linear, expanding the difference is the same as differencing the expansions:

	SHA-0	SHA-1
difference bits over 80 rounds	31	109
distinct bit positions ever touched	**1**	**23**
mask of positions touched	`00000002`	`00fffffe`

2.2 Proved

Measurement shows it for one input. `broken/lean/Sha01/Expansion.lean` proves it for all of them. The statement is commutation with masking: masking a word keeps only the bit positions the mask selects, so

$$\text{expand}(\text{mask } M) = \text{mask}(\text{expand } M)$$

says precisely "the expansion does not move information between bit positions". In Lean:

```
theorem sha0_expand_mask (M : Nat -> BitVec 32) (m : BitVec 32) :
    forall t, W M 0 t &&& m = W (fun k => M k &&& m) 0 t
```

The corresponding statement for SHA-1 is refuted by explicit counterexample, in `sha1_expand_not_mask`. Both proofs are kernel-only — they depend on `propext` and `Quot.sound` and nothing else, not even `Classical.choice`, and deliberately not on a SAT-backed tactic that would put the Lean compiler in the trusted base.

2.3 What that buys the attacker

Under (4) a difference pattern in one bit column is determined by its first sixteen bits, so the possible patterns form a $[80, 16]$ binary linear code with exactly 65 536 codewords. `expansion code` enumerates all of them in about a tenth of a second and reports the minimum weight: **25** for a codeword usable in a full 80-round attack. Under (2) the corresponding space is not a per-column code at all and cannot be enumerated.

3 Local collisions

A *local collision* is the atom of the attack. Flip bit i of W_t. The difference enters register a at round t; the next five message words can be chosen to cancel it before it escapes into the chaining value.

The cancellation is not free. The round adds modulo 2^{32} while the differential is stated in XOR, and f_t may or may not pass the difference through. `collide verify` measures how often it works, over 200 000 random states per row:

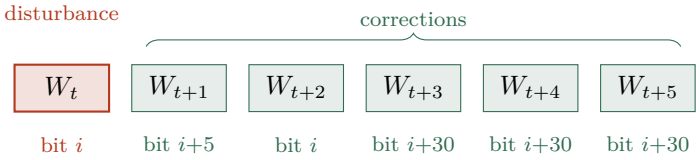

Figure 2: One local collision. The offsets are not arbitrary: +5 undoes the $\mathrm{ROTL}^5(a)$ of the following round, and the three +30s track the bit as ROTL^{30} carries it through register c (Figure 1). The same six-word pattern is what `add_local_collision` in `broken/attack/collide.c` writes.

rounds	f_t	$i = 1$	$i = 2$
0–19	Ch	0.0157	0.0027
20–39	Parity	**0.1242**	0.0167
40–59	Maj	0.0152	0.0029
60–79	Parity	**0.1247**	0.0169

Two things follow, and both were corrected *by* the measurement rather than confirmed by it.

The Parity rounds are cheaper but not free. It is often said that a linear f makes a local collision cost nothing. The measurement says $\approx 2^{-3}$, about eight times cheaper than the nonlinear rounds and not free at all. The reason is worth stating precisely: making f linear removes the conditions *on* f, and what is left is the additions. Equation (3) adds modulo 2^{32}, so an XOR correction is exact only when no carry crosses the corrected bit. That residual is the 2^{-3}.

Bit 1 is the right place to work. Bit 1 beats bit 2 by the same factor of eight, for the same reason: the corrections sit at $i + 5$ and $i + 30$, and at $i = 1$ the $i + 30$ corrections land at bit 31, where a carry out of the top is discarded instead of propagating. Every disturbance vector in the literature uses bit 1, and this is why.

4 Disturbance vectors, and a condition that is easy to miss

A single local collision is not a collision of the hash: the message difference it needs must itself be producible by the expansion. So the attacker picks a codeword of the code in §2.3 — the *disturbance vector* — and places one local collision at every round where it is set. Its weight is the number of local collisions to be paid for, so low weight is the whole game, and the last five rounds must be quiet because a disturbance at round $R - 1$ cannot be corrected before the block ends.

There is a further condition, and omitting it is not a small mistake. The message difference in a given bit column is a sum of *time-shifted* copies of the disturbance vector, one per correction offset. Every one of those shifts has to be a codeword in its own right. A shift by k satisfies (4) only from round $16 + k$ onwards, so requiring shifts 1 through 5 to be codewords imposes $1 + 2 + 3 + 4 + 5 = 15$ extra linear conditions.

That is not vacuous: it cuts the 65 535 nonzero codewords down to **2047**, a subspace of dimension 11. The first version of `broken/attack/collide.c` left the condition out and reported `difference is a valid expansion: NO` for every round count — the differential was not merely unlikely, it was unmountable. With the condition imposed the same program finds collisions immediately.

5 What actually runs here

`collide search` builds the message difference from a disturbance vector and searches for a message pair on which every local collision happens to cancel. Both messages are exactly one

64-byte block, so they pad identically and a collision in the first compression is a collision of the whole hash — not merely of the compression function.

rounds	DV weight	result	time
20	3	collision	< 0.1 s
25	3	collision	< 0.1 s
30	7	not found in 25 s	—
40	12	not found in 25 s	—

A collision found on the machine this was written on, and re-verified through the command-line tool rather than trusted from the search:

```
m1 = a0ff9c057ffb967705882f6996497503caa75b02710d381205465e6257897d37
     d8e7937059d6088b323f31b7986989ed0ba3ef5dc0d1bf15bb2ffd82420a7f32
m2 = a0ff9c057ffb967705882f6996497503caa75b02710d381205465e6057897d77
     d8e79372d9d6088bb23f31b7186989ed0ba3ef5dc0d1bf15bb2ffd80420a7f72

SHA0-25(m1) = 6b967bd82efc21d8011755a15a10a2e7200a9094
SHA0-25(m2) = 6b967bd82efc21d8011755a15a10a2e7200a9094
```

At full 80 rounds the same two messages hash to `9a276e8d...` and `ccbaa7e1...`: different, as they must be.

Beyond about 25 rounds the search needs *message modification* — fixing up the early rounds deterministically instead of waiting for random messages to satisfy them — which is what Wang's work supplied and what this implementation does not have. That is the honest boundary of what is here.

6 The history

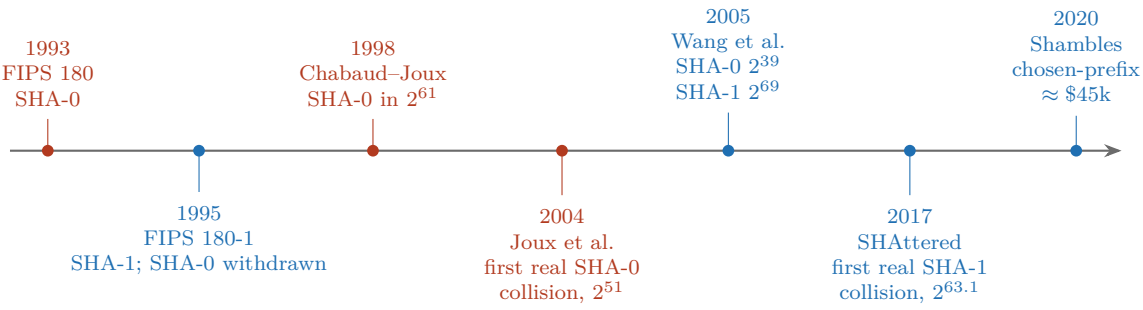

Figure 3: Twenty-seven years. The gap between the two red clusters and the two blue ones is what the rotation of (2) bought.

1993–1995. NIST published SHA-0, then withdrew it two years later and replaced it with SHA-1, saying only that a design flaw had been found. No technical justification was published at the time.

1998. Chabaud and Joux introduced local collisions and disturbance vectors and gave a 2^{61} attack on the SHA-0 compression function [3]. Their abstract already records the punchline of this document: the same method "is unable to find collisions faster than the birthday paradox" for SHA-1, which they call strong evidence that the rotation was added for exactly this reason.

2004. Joux, with Carribault, Lemuet and Jalby, found the first actual SHA-0 collision on a 256-CPU machine [4].

2005. Wang, Yu and Yin reduced SHA-0 to about 2^{39} [5] and, with the same machinery, gave the first attack on full SHA-1 below the birthday bound at about 2^{69} [6]. SHA-1 was theoretically broken from that moment and remained in wide deployment for another twelve years.

2017. SHAttered produced two PDF files with the same SHA-1, at a cost of roughly $2^{63.1}$ compressions [7].

2020. Leurent and Peyrin gave the first *chosen-prefix* collision for around $45,000 of rented GPU time [8]. A chosen-prefix collision is the one that matters operationally: it lets an attacker collide two documents each of which begins however they like.

7 What is not here

- **No full SHA-0 or SHA-1 collision.** Those cost about 2^{39} and 2^{63} and are not reachable on one machine in one session. Nothing in `broken/` claims otherwise.
- **No message modification.** §5 stops around 25 rounds for want of it.
- **No historic collision data.** The SHAttered blocks are not embedded; the paper is cited and mirrored instead.
- **No SHA-0 oracle exists.** SHA-1 is checked here against three independent implementations. SHA-0 is implemented by nothing current, so it rests on published known-answer vectors plus the structural check that switching the rotation on turns every SHA-0 answer into the oracle-verified SHA-1 one.
- **Neither function is safe to use.** That is the point of the document.

References

[1] National Institute of Standards and Technology. *Secure Hash Standard*. Tech. rep. FIPS PUB 180. The original SHA, retrospectively called SHA-0. Withdrawn in 1995. **No local copy**: NIST does not serve the withdrawn 1993 publication, and no authoritative PDF was located. The algorithm as implemented here is FIPS 180-1 with the expansion rotation removed, which is the documented relationship between the two. U.S. Department of Commerce, May 1993 (cit. on p. 1).

[2] National Institute of Standards and Technology. *Secure Hash Standard*. Tech. rep. FIPS PUB 180-1. SHA-1: FIPS 180 plus a one-bit rotation in the message expansion. **No local copy**: the archived PDF returned 404 from every NIST path tried on 2026-08-10. The superseding FIPS 180-4 [9] is mirrored in `refs/` and specifies SHA-1 identically. U.S. Department of Commerce, Apr. 1995. URL: https://csrc.nist.gov/pubs/fips/180-1/final (cit. on p. 1).

[3] Florent Chabaud and Antoine Joux. "Differential Collisions in SHA-0". In: *Advances in Cryptology — CRYPTO '98*. Vol. 1462. Lecture Notes in Computer Science. Local copy: `refs/chabaud-joux-1998.pdf`, retrieved 2026-08-10 via the Internet Archive's copy of the publisher PDF. Identity confirmed from the title page. This is the paper that introduced local collisions and disturbance vectors, and its abstract already records that the same method fails against SHA-1 — the earliest statement of the point this directory demonstrates by measurement. Springer, 1998, pp. 56–71. DOI: 10.1007/BFb0055720 (cit. on p. 5).

[4] Eli Biham, Florent Chabaud, and Antoine Joux. "Collisions of SHA-0". In: *Announcement at CRYPTO 2004 rump session*. The first actual SHA-0 collision, found by Joux with Carribault, Lemuet and Jalby on a 256-CPU machine. **No local copy**: a rump-session announcement, never published as a paper with a stable PDF. Cited for the historical fact and the cost figure only, both of which are corroborated by [5]. 2004 (cit. on p. 5).

[5] Xiaoyun Wang, Hongbo Yu, and Yiqun Lisa Yin. “Efficient Collision Search Attacks on SHA-0”. In: *Advances in Cryptology — CRYPTO 2005*. Vol. 3621. Lecture Notes in Computer Science. **No local copy**: publisher PDF is paywalled and no author-hosted copy was reachable on 2026-08-10. Cited for the reduction of SHA-0 collision cost to roughly 2^{39}. Springer, 2005, pp. 1–16. DOI: 10.1007/11535218_1 (cit. on pp. 5, 6).

[6] Xiaoyun Wang, Yiqun Lisa Yin, and Hongbo Yu. “Finding Collisions in the Full SHA-1”. In: *Advances in Cryptology — CRYPTO 2005*. Vol. 3621. Lecture Notes in Computer Science. **No local copy**: paywalled, as above. The first attack on full SHA-1 below the birthday bound, at about 2^{69}. Springer, 2005, pp. 17–36. DOI: 10.1007/11535218_2 (cit. on p. 5).

[7] Marc Stevens et al. “The First Collision for Full SHA-1”. In: *Advances in Cryptology — CRYPTO 2017*. Vol. 10401. Lecture Notes in Computer Science. Local copy: `refs/shattered-2017.pdf`, retrieved 2026-08-10 from shattered.io. Identity confirmed from the title page. The SHAttered collision: two PDFs with the same SHA-1, at a cost of roughly $2^{63.1}$ compressions. Springer, 2017, pp. 570–596. DOI: 10.1007/978-3-319-63697-9_19. URL: https://shattered.io/static/shattered.pdf (cit. on p. 6).

[8] Gaëtan Leurent and Thomas Peyrin. “SHA-1 is a Shambles: First Chosen-Prefix Collision on SHA-1 and Application to the PGP Web of Trust”. In: *29th USENIX Security Symposium*. **No local copy**: the IACR ePrint server returned 403 to every automated request made on 2026-08-10, and the project site hosts no PDF at a guessable path. The chosen-prefix collision, at about $2^{63.4}$ and roughly $45,000 of rented GPU time — the result that made SHA-1 forgery practical rather than merely demonstrable. 2020, pp. 1839–1856. URL: https://eprint.iacr.org/2020/014 (cit. on p. 6).

[9] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Tech. rep. FIPS PUB 180-4. Local copy: `refs/fips180-4.pdf`, retrieved 2026-08-09. Carries the current normative text of SHA-1 as well as the SHA-2 family. U.S. Department of Commerce, 2015. DOI: 10.6028/NIST.FIPS.180-4 (cit. on p. 6).